# SCoPe: Trust-Region Latent Planning with JEPA World Models

22AIE401 Reinforcement Learning

Aayush Shenday
AM.SC.U4AIE23004
Amrita School of Computing
Amritapuri

Rajashekar Kasarla
AM.SC.U4AIE23041
Amrita School of Computing
Amritapuri

Dr. Geeta
Amrita School of Computing
Amritapuri

***Abstract*—A robot can plan by imagining the result of many possible action sequences with a learned *world model* and choosing the one whose imagined result is closest to the goal. Such planners trust every imagined future equally, even when the model's prediction is wrong. We present SCoPe, a planner that first checks how reliable each imagined step is and rejects plans that rely on unreliable predictions. We build on DINO-WM, a world model that predicts the future in the feature space of a pretrained image encoder. First, we reproduce DINO-WM's planning results on a single 8 GB laptop GPU, with code changes that make planning 3.8× faster without changing any of its decisions. We reach success rates of 0.900 on Push-T, 0.947 on Wall and 1.000 on PointMaze, against published values of 0.90, 0.96 and 0.98. Second, we compare five reliability scores on 76,800 imagined steps whose real outcome we obtained from the simulator. A score based on Monte-Carlo dropout identifies the most wrong predictions with an AUROC of 0.86, clearly better than random guessing (0.50) and a simple baseline (0.79), on all three seeds. Third, we show that removing unreliable plans only at the final selection step hardly changes success, so the check must be used throughout the planning process.**

***Index Terms*—world models, model-based reinforcement learning, planning, uncertainty estimation**

## I. Introduction

In model-based reinforcement learning, an agent learns a *world model*: a neural network that predicts what will happen next if the agent takes a given action. The agent can then plan by trying many action sequences "in its head" and executing the most promising one. DINO-WM [1] is a recent world model of this kind. It describes each camera image with the features of a pretrained image encoder, DINOv2 [3], and learns to predict how these features change after an action. This design, in which the model predicts features rather than pixels, is called a joint-embedding predictive architecture (JEPA) [5]. DINO-WM plans without any reward function: it only needs an image of the goal, and it reaches 90% success on the Push-T pushing task.

The weakness of such planners is that they never ask whether the model's imagination is correct. Fig. 1 shows a case from our own runs. The model imagines the T-shaped block moving onto the goal, the planner commits to this plan, and in the simulator the block barely moves. The episode fails because the planner trusted a wrong prediction.

SCoPe addresses this weakness. Before the planner chooses a plan, SCoPe gives every imagined step a *trust score* that estimates how likely the prediction is to be wrong, and rejects plans that contain untrusted steps. The planner then chooses only among plans it can trust. The name refers to this idea: planning stays inside the *trust region*, the set of situations in which the model's predictions are reliable.

**Novel contributions.**

1) *Trust-checked planning.* We add a reliability check to the planner of a frozen world model. The world model is not retrained, so the method can be added to any similar model.
2) *A way to measure prediction errors.* We execute the planner's candidate plans in the simulator and compare each imagined step with what really happened. This gives, for every imagined step, its true error.
3) *A trust score that works without training.* Of the five scores we compare, Monte-Carlo dropout finds the most wrong predictions best (AUROC 0.86), on every seed. The planner's own confidence carries no such information.
4) *Where the check must be applied.* Removing unreliable plans only at the last step barely changes success, even if the true errors are known. The check must therefore act during the whole search.
5) *An efficient reproduction.* DINO-WM runs on a single 8 GB laptop GPU, 3.8× faster than the original code, with identical planning decisions.

## II. Related Work

**Uncertainty in model-based RL.** Several methods avoid trusting a learned model where it is uncertain. PETS [6] plans with several models and measures how much they disagree; MBPO [8] only uses short imagined rollouts; MOPO [9] and MOReL [10] penalise uncertain states when learning a policy from fixed data. These methods are used while *training* a policy, mostly with simple state vectors rather than images.

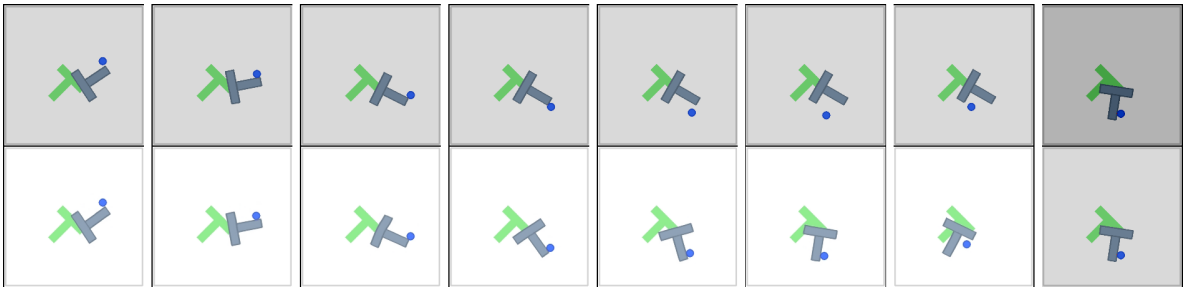


Fig. 1. A failure from our runs (Push-T). Top: what happened in the simulator. Bottom: what DINO-WM imagined for the same actions. In the imagination the block reaches the goal; in reality it barely moves. SCoPe is designed to detect such wrong predictions before a plan is chosen.

**World models that plan from images.** PlaNet [7], DreamerV3 [12] and TD-MPC2 [13] learn compact models of images together with a reward. DINO-WM [1] and V-JEPA 2-AC [2] plan towards a goal image without a reward, but they trust every imagined future equally.

**Known difficulties.** Measuring uncertainty is not easy: disagreement between models can miss errors that build up over time [17], and penalising uncertainty can even make results worse [18]. We therefore test every trust score against the true error instead of assuming that it works.

**Our difference.** SCoPe checks the reliability of each imagined step *at the moment of planning*, for a frozen image-based world model, and uses the check to reject plans before one is chosen.

## III. Problem Formulation

The agent sees an image of the scene and its own position. The encoder turns the image into a feature vector $z$, the *latent state*. An action is a short movement of the agent. Given the current latent state and a sequence of actions, the world model predicts the future latent states $\hat{z}_1, \ldots, \hat{z}_5$, five steps ahead. The goal is given as an image, with latent state $z_g$. An episode is a success if the simulator reports that the goal was reached.

The DINO-WM planner chooses the action sequence $a$ whose last imagined state is closest to the goal, $C(a) = \|\hat{z}_5 - z_g\|^2$. SCoPe adds a condition: every imagined step must be trusted,

$$\min_a C(a) \quad \text{subject to} \quad u(\hat{z}_k) \le \tau \ \text{ for } k = 1, \ldots, 5, \quad (1)$$

where $u$ is a trust score (a higher value means less trusted) and $\tau$ is a threshold. This is a sequential decision problem, because the effect of an action appears several steps later and success is only known at the end.

## IV. Method

### A. Planning with the cross-entropy method

DINO-WM plans with the cross-entropy method (CEM), a simple search procedure. CEM samples 300 random action sequences, imagines the result of each one with the world model, keeps the 30 best, and samples new sequences around them. After 30 such iterations it executes the beginning of the best sequence, observes the new image, and plans again. This repeated planning is called model-predictive control (MPC); one plan-and-execute cycle is one *round*.

**Algorithm 1** SCoPe planning round

observe the image and encode it
**for** each CEM iteration **do**
  sample 300 action sequences
  imagine the result of each one with the world model
  compute the trust score of every imagined step
  keep only sequences whose steps are all trusted
  **if** too few sequences are kept **then**
    fall back (wider search, shorter plan, safe action)
  **end if**
  refit the sampling to the 30 lowest-cost kept sequences
**end for**
execute the start of the best sequence; repeat

### B. SCoPe

Fig. 2 shows the system and Algorithm 1 the decision loop. SCoPe adds two parts to the planner: a *trust module*, which scores every imagined step of every candidate, and a *gate*, which removes candidates that contain an untrusted step. Only the remaining candidates compete on cost. If too few candidates pass, SCoPe falls back to a wider search, then to a shorter plan, then to a safe action. The world model itself is never changed.

### C. Trust scores

We compare five scores. None of them needs any training:

- **MC-dropout** [23]: the world model contains dropout, which randomly switches off parts of the network. We run the prediction four times with dropout switched on. If the four predictions disagree, the model is uncertain.
- **Action perturbation**: how much the prediction changes when the actions are changed slightly.
- **Image perturbation**: how much the prediction changes when the input image is changed slightly.
- **Distance to training data**: how far the imagined state is from the states seen during training.
- **Imagined cost**: the planner's own estimate of how far the plan ends from the goal.

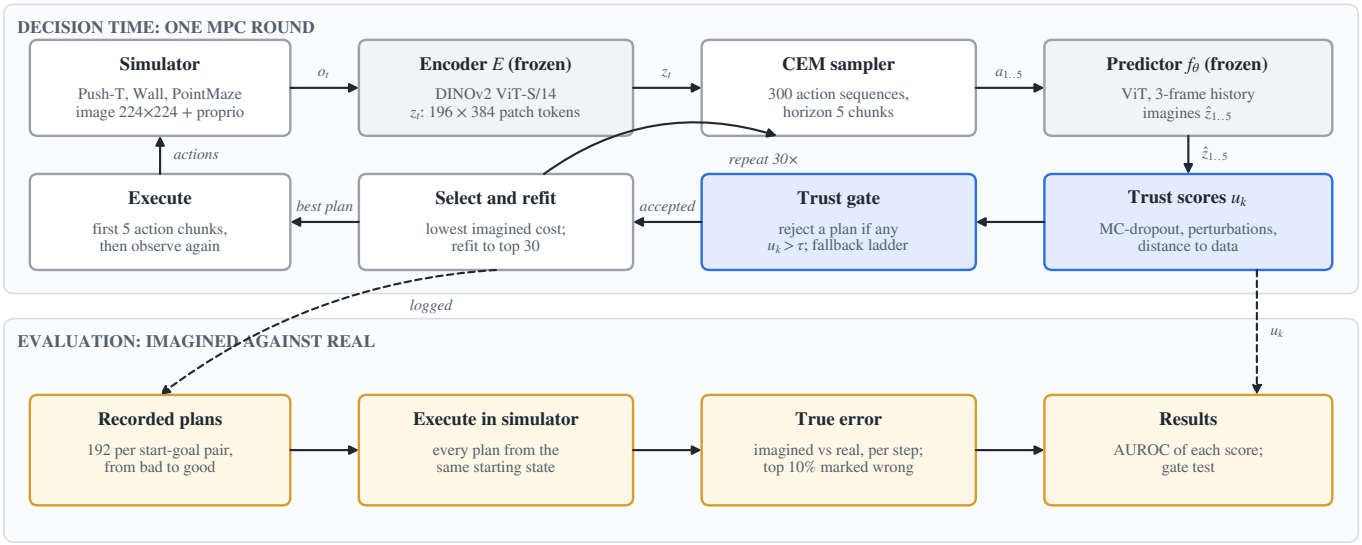



Fig. 2. SCoPe architecture. Top: one planning round. The frozen DINO-WM encoder and predictor (grey) imagine every candidate plan; the trust module (blue) scores each imagined step, and the gate removes untrusted plans before the best one is selected. Bottom: how we evaluate the trust scores, by executing recorded plans in the simulator and comparing imagined and real outcomes.

We compare them with two baselines: a **random** score, and **distance moved**, which is simply how far the imagined state is from the starting state.

### D. Evaluation questions

**Q1:** Can DINO-WM be reproduced on a single 8 GB GPU? **Q2:** Which trust score best identifies the imagined steps that are really wrong, and is it better than both baselines? **Q3:** Does removing untrusted plans make the chosen plan succeed more often?

## V. Experimental Setup

**Tasks.** We use the three simulated tasks of DINO-WM: Push-T (push a T-shaped block onto a target), Wall (move through a door in a wall) and PointMaze (move a ball through a maze), with the published datasets and trained models [26]. For each task we run 50 start–goal pairs per seed, with seeds 99, 100 and 101, and allow up to three planning rounds.

**Hardware.** All experiments run on one RTX 5060 laptop GPU with 8 GB of memory. The original code did not fit in this memory and needed 26.2 s for one batch of imagined rollouts. We optimised the code (for example, by encoding the shared starting image once instead of 300 times), which reduced this to 6.9 s, a 3.8× speed-up. We checked that the planner selects exactly the same plans as before.

**Measuring the true error.** On 80 Push-T start–goal pairs over 3 seeds, we recorded 192 of the plans that CEM considered for each pair, from bad early plans to good late ones. We imagined each plan with the world model, and then executed the same plan in the simulator from the same starting state. Comparing the imagined and the real latent state after each step gives the true prediction error of 76,800 imagined steps. We mark the 10% of steps with the largest error as *wrong*.

TABLE I
Planning success over 50 start–goal pairs per seed (mean ± standard deviation over seeds).

| Task | Seeds | One plan (round 1) | With replanning | Paper [1] |
|---|---|---|---|---|
| Push-T | 3 | 0.87 ± 0.05 | 0.90 ± 0.05 (round 3) | 0.86 / 0.90 |
| Wall | 3 | 0.52 ± 0.02 | 0.95 ± 0.02 (round 3) | 0.74 / 0.96 |
| PointMaze | 1 | 0.86 | 1.00 (round 2) | 0.80 / 0.98 |

**Metric.** We measure each trust score with the AUROC: the probability that a randomly chosen wrong step receives a higher score than a randomly chosen correct step. An AUROC of 0.5 means random guessing and 1.0 means perfect detection. The 95% confidence intervals are computed by bootstrap resampling over the start–goal pairs.

**Gate test.** For each start–goal pair we take the recorded plans, remove the 30% least trusted ones, and choose the plan with the lowest imagined cost. We then check whether this plan succeeds in the simulator. For comparison we also remove plans at random, and with an "oracle" that knows the true errors.

## VI. Results

### A. Q1: Reproduction of DINO-WM

Table I and Fig. 3 show the results. After replanning, our success rates match the published ones: 0.900 against 0.90 on Push-T, 0.947 against 0.96 on Wall, and 1.000 against 0.98 on PointMaze. With a single plan, Push-T (0.873) and PointMaze (0.860) are slightly above the published values. Wall with a single plan is lower (0.520 against 0.74), consistently over the three seeds; replanning brings it to the published level. Fig. 4 shows how the planner improves its plan over the CEM iterations: most of the gain comes in the first ten iterations.

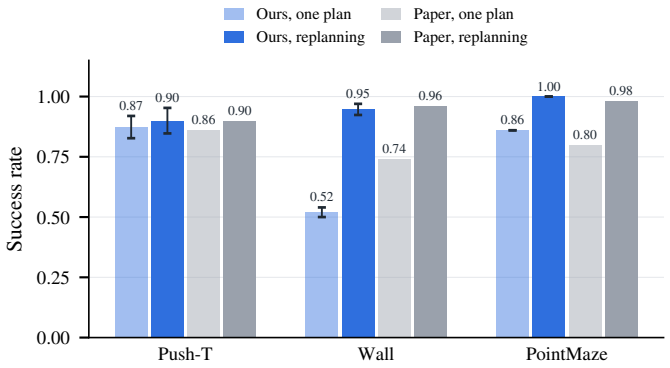



Fig. 3. Planning success after one plan and after replanning: ours (blue, mean ± standard deviation over seeds) and the published DINO-WM results (grey).

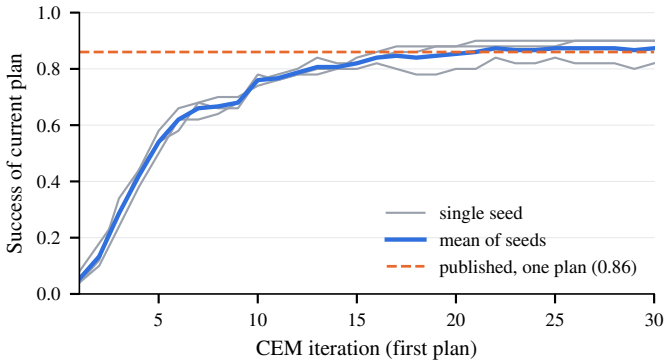



Fig. 4. Push-T: success rate of the planner's current plan after each CEM iteration, for each seed (grey) and on average (blue). Dashed: published result for a single plan.

### B. Q2: Which trust score detects wrong predictions?

Table II and Fig. 5 show the results. **MC-dropout is the best score**, with an AUROC of 0.86 (95% interval 0.84–0.88). It is better than random guessing (0.50) and better than the simple baseline (0.79), and this holds on each of the three seeds (0.87, 0.84, 0.88). The simple baseline is surprisingly strong: plans that move the scene a lot are more often predicted wrongly. This is why a useful trust score must beat this baseline, not only random guessing. Action perturbation (0.66) and distance to the training data (0.62) carry some information but less than the baseline. The planner's own imagined cost carries none (0.51): a plan that *looks* good to the planner is not more likely to be predicted correctly. Image perturbation is at the level of random guessing (0.50), most likely because the image encoder ignores small changes to the input image.

### C. Q3: Does removing untrusted plans help?

Table III shows the result. Without removal, the chosen plan succeeds for 71.3% of the start–goal pairs; with MC-dropout removal, for 70%. One pair corresponds to 1.3%, so these differences are within noise. Even the oracle, which knows the true errors, only reaches 72.5%. The reason is that the plan with the lowest imagined cost is rarely one of the badly predicted plans, so removing plans at the very end changes little. Detecting wrong predictions is therefore necessary but not enough: to improve success, the trust check has to guide the whole search, in every CEM iteration and every round, as in Algorithm 1.

TABLE II
HOW WELL EACH SCORE DETECTS THE 10% MOST WRONG IMAGINED STEPS (AUROC; 0.5 = RANDOM GUESSING), 80 PUSH-T START–GOAL PAIRS, 3 SEEDS.

| Trust score | AUROC | 95% interval |
|---|---|---|
| **MC-dropout** | **0.863** | **0.84–0.88** |
| Action perturbation | 0.657 | 0.64–0.67 |
| Distance to training data | 0.619 | 0.56–0.66 |
| Imagined cost | 0.511 | 0.47–0.55 |
| Image perturbation | 0.496 | 0.44–0.55 |
| Distance moved (baseline) | 0.790 | 0.76–0.82 |
| Random (baseline) | 0.500 | 0.49–0.51 |

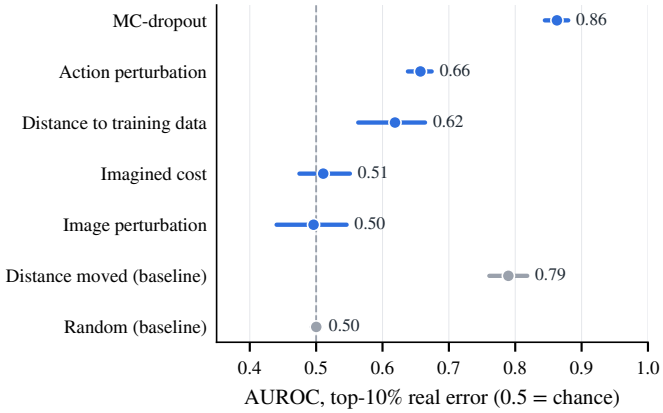



Fig. 5. AUROC of each score with its 95% confidence interval. Blue: trust scores; grey: baselines; dashed line: random guessing.

## VII. Discussion and Limitations

The results support the main idea of SCoPe: the world model's own dropout already shows where its predictions are unreliable, at the cost of four extra predictions per plan and with no training. They also give two lessons for designing such a check. First, a trust score must be compared with a simple baseline, not only with random guessing. Second, filtering plans only at the final choice is not enough.

The trust scores and the gate test use the Push-T task. All tasks are simulated, and planning takes minutes per decision on our GPU, so the system does not yet run in real time. Using it on a real robot would require a world model trained on robot data, such as V-JEPA 2-AC [2].

## VIII. Conclusion

We presented SCoPe, a planner that checks the reliability of a world model's predictions before acting on them. We reproduced DINO-WM on a single 8 GB GPU, with success rates of 0.900 on Push-T, 0.947 on Wall and 1.000 on PointMaze, and made planning 3.8× faster. Across 76,800 imagined steps compared with the simulator, Monte-Carlo

TABLE III
SUCCESS OF THE CHOSEN PLAN AFTER REMOVING 30% OF THE RECORDED PLANS, 80 START–GOAL PAIRS.

| Plans removed by | Chosen plan succeeds |
|---|---|
| No removal | 71.3% |
| Oracle (knows the true error) | 72.5% |
| Random | 70% |
| MC-dropout | 70% |
| Action perturbation | 75% |
| Distance to training data | 68.8% |
| Distance moved | 67.5% |

dropout detected the most wrong predictions with an AUROC of 0.86, better than random guessing and a simple baseline on every seed, while the planner's own confidence gave no such signal. Finally, we showed that the check must be used during the whole planning search rather than only at the final choice.

**Future work.** Using the trust check inside every CEM iteration on all three tasks, combining several scores, and testing under changed visual conditions.

## REPRODUCIBILITY

The code, run scripts and report sources are in the project repositories (scope-planning, scope-project, scope-mid-review). Every number in the tables and text is generated from the run logs by `report/make_assets.py`.

## REFERENCES


[1] G. Zhou, H. Pan, Y. LeCun, L. Pinto. DINO-WM: World Models on Pre-trained Visual Features enable Zero-shot Planning. ICML 2025, PMLR 267:79115–79135. https://arxiv.org/abs/2411.04983

[2] M. Assran et al. V-JEPA 2: Self-Supervised Video Models Enable Understanding, Prediction and Planning. arXiv:2506.09985, 2025. https://arxiv.org/abs/2506.09985

[3] M. Oquab et al. DINOv2: Learning Robust Visual Features without Supervision. TMLR, 2024. https://arxiv.org/abs/2304.07193

[4] M. Assran et al. Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture (I-JEPA). ICCV 2023. https://arxiv.org/abs/2301.08243

[5] Y. LeCun. A Path Towards Autonomous Machine Intelligence. Position paper, v0.9.2, 2022 (not peer-reviewed). https://openreview.net/forum?id=BZ5a1r-kVsf

[6] K. Chua, R. Calandra, R. McAllister, S. Levine. Deep Reinforcement Learning in a Handful of Trials using Probabilistic Dynamics Models (PETS). NeurIPS 2018. https://arxiv.org/abs/1805.12114

[7] D. Hafner et al. Learning Latent Dynamics for Planning from Pixels (PlaNet). ICML 2019, PMLR 97:2555–2565. https://arxiv.org/abs/1811.04551

[8] M. Janner, J. Fu, M. Zhang, S. Levine. When to Trust Your Model: Model-Based Policy Optimization (MBPO). NeurIPS 2019. https://arxiv.org/abs/1906.08253

[9] T. Yu et al. MOPO: Model-based Offline Policy Optimization. NeurIPS 2020. https://arxiv.org/abs/2005.13239

[10] R. Kidambi, A. Rajeswaran, P. Netrapalli, T. Joachims. MOReL: Model-Based Offline Reinforcement Learning. NeurIPS 2020. https://arxiv.org/abs/2005.05951

[11] B. Frauenknecht, A. Eisele, D. Subhasish, F. Solowjow, S. Trimpe. Trust the Model Where It Trusts Itself: Model-Based Actor-Critic with Uncertainty-Aware Rollout Adaption (MACURA). arXiv:2405.19014, 2024. https://arxiv.org/abs/2405.19014

[12] D. Hafner, J. Pasukonis, J. Ba, T. Lillicrap. Mastering Diverse Control Tasks through World Models (DreamerV3). Nature 640:647–653, 2025. https://doi.org/10.1038/s41586-025-08744-2

[13] N. Hansen, H. Su, X. Wang. TD-MPC2: Scalable, Robust World Models for Continuous Control. ICLR 2024. https://arxiv.org/abs/2310.16828

[14] V. Sobal et al. Learning from Reward-Free Offline Data: A Case for Planning with Latent Dynamics Models (PLDM). NeurIPS 2025. https://arxiv.org/abs/2502.14819

[15] W. Zhang et al. Hierarchical Planning with Latent World Models (HWM). arXiv:2604.03208, 2026. https://arxiv.org/abs/2604.03208

[16] J. Seo, K. Nakamura, A. Bajcsy. Uncertainty-aware Latent Safety Filters for Avoiding Out-of-Distribution Failures. CoRL 2025. https://arxiv.org/abs/2505.00779

[17] J. Berger, B. Frauenknecht, S. Trimpe, B. Leibe. Biased Dreams: Limitations to Epistemic Uncertainty Quantification in Latent Dynamics Models. RLC 2026. https://arxiv.org/abs/2604.25416

[18] Z. Wang. Learning from World Feedback: Why Model Uncertainty Fails as a Risk Signal in Model-Based RL. ICML 2026 Workshop on RL from X Feedback. https://arxiv.org/abs/2607.16591

[19] G. An et al. Diagnosing JEPA World Models with Action-Conditioned Predictive Consistency (ACPC). arXiv:2608.12939, 2026. https://arxiv.org/abs/2608.12939

[20] J. Hu et al. Representation Geometry Matters for Planning with JEPA World Models. arXiv:2608.16287, 2026. https://arxiv.org/abs/2608.16287

[21] G. Lee, B.-T. Zhang. Beyond Task Success: Stage-Wise Reliability of World Model Planning under Sensing Degradation. arXiv:2609.07126, 2026. https://arxiv.org/abs/2609.07126

[22] B. Lakshminarayanan, A. Pritzel, C. Blundell. Simple and Scalable Predictive Uncertainty Estimation using Deep Ensembles. NIPS 2017. https://arxiv.org/abs/1612.01474

[23] Y. Gal, Z. Ghahramani. Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning. ICML 2016. https://arxiv.org/abs/1506.02142

[24] C. Chi et al. Diffusion Policy: Visuomotor Policy Learning via Action Diffusion. RSS 2023 (origin of the Push-T task used by DINO-WM). https://arxiv.org/abs/2303.04137

[25] J. Fu, A. Kumar, O. Nachum, G. Tucker, S. Levine. D4RL: Datasets for Deep Data-Driven Reinforcement Learning. arXiv:2004.07219, 2020 (PointMaze). https://arxiv.org/abs/2004.07219

[26] DINO-WM code (MIT licence), pinned at commit 0a9492f; datasets and checkpoints on OSF (project bmw48). https://github.com/gaoyuezhou/dino_wm